Sarabo — বাংলায় পুরো Project বোঝা

নিজের প্রস্তুতির জন্য, formal report না। উদ্দেশ্য একটাই: কেউ জিজ্ঞেস করলে যেন গুছিয়ে, পরিষ্কার করে বলতে পারি project-টা কী, কেন, কীভাবে কাজ করে। সব সংখ্যা report আর code থেকে মিলিয়ে লেখা। ২৪ অক্টোবর ২০২৬।

> **এক লাইনে:** Sarabo হলো একটা web platform, যেখানে customer অনলাইনে phone/laptop/TV/fridge/AC ইত্যাদি repair-এর request দেয়, এলাকার verified technician-রা offer পাঠায়, customer একজনকে বেছে নেয়, আর device তুলে নেওয়া থেকে repair, payment, ফেরত দেওয়া, warranty পর্যন্ত সবকিছু track করা যায়।

১. সমস্যাটা কী ছিল? (কেন বানালাম)

আমাদের দেশে কিছু নষ্ট হলে সাধারণত যা হয়:

- দোকান খুঁজি পরিচিত কারো কাছ থেকে শুনে — দোকানটা ভালো কিনা জানার উপায় নেই।
- দাম আগে থেকে জানি না। device দিয়ে আসার পর দাম বলে, তখন দর কষাকষি করার সুযোগ কম।
- device এখন কোথায়, কী অবস্থায় — কিছুই দেখতে পাই না, বারবার ফোন করতে হয়।
- কোনো লিখিত record নেই। repair-এর পর আবার নষ্ট হলে প্রমাণ বা warranty দাবি করার কিছু নেই।

Sarabo এই informal কাজটাকে একটা **tracked, দাম-জানা, record-রাখা service**-এ পরিণত করে। আগে দাম জানো, technician বেছে নাও, প্রতিটা ধাপ দেখো, টাকা নিরাপদে দাও, আর কিছু হলে warranty আছে।

২. কারা ব্যবহার করে? (৩টা role)

Role	কী করে
Customer	repair request দেয়, offer গুলো তুলনা করে একজনকে বেছে নেয়, inspection fee ও repair-এর টাকা দেয়, progress দেখে, শেষে review বা private report দেয়।
Technician	আগে apply করে (কোন device, কোন repair পারে, কত বছরের অভিজ্ঞতা, কোন এলাকা); admin approve করলে jobs board থেকে কাজ দেখে offer পাঠায়, device তোলে, inspect করে, final quote দেয়, repair করে, wallet-এ টাকা পায়, bKash-এ তোলে।
Admin	technician approve করে, service-এর তালিকা আর দাম ঠিক করে, আটকে থাকা কাজ (overdue pickup, offer না আসা request) দেখে, refund আর withdrawal সামলায়, review/report moderate করে, report দেখে।
Public visitor	login ছাড়াই service list, service area map, open jobs board আর tracking code দিয়ে repair-এর অবস্থা দেখতে পারে।

Database-এ role-গুলোর নাম `user`, `rider`, `admin` — কারণ project-টা একটা পুরনো parcel-delivery codebase থেকে শুরু হয়েছিল। ব্যবহারকারী এগুলো কখনো দেখে না।

৩. পুরো কাজটা কীভাবে চলে — রহিমের ফোনের গল্প

ধরো রহিমের ফোনের screen ফেটে গেছে। Sarabo-তে যা হয়:

1. **Request দেওয়া:** রহিম device আর service (যেমন "screen replacement") বেছে নেয়। সাথে সাথে catalogue থেকে আনুমানিক দাম দেখায়। সমস্যা লিখে দেয় (চাইলে বাংলা/Banglish-এ লিখলে AI form পূরণ করে দেয়), সর্বোচ্চ ৩টা ছবি দেয়, আর একটা ২ ঘণ্টার pickup slot বাছে। রহিমের এলাকায় এই কাজ পারে

এমন কোনো approved technician না থাকলে request-টাই নেওয়া হয় না — যাতে কেউ শুধু শুধু অপেক্ষা না করে।

2. **Offer আসা:** request-টা jobs board-এ চলে যায়, মিলে যায় এমন technician-রা notification পায়। তারা offer পাঠায়: আনুমানিক দামের range + inspection fee + একটা note।
3. **বেছে নেওয়া:** রহিম offer গুলো দেখে। সবচেয়ে ভালোটার উপর "Best match" badge থাকে (নিচে smarter matching দেখো)। একজনকে বেছে inspection fee Stripe দিয়ে দেয় — payment হলেই technician book হয়ে যায়। পছন্দ না হলে reject করা যায়, তখন job আবার post হয়।
4. **Pickup ও inspection:** technician এসে ফোন নেয়, ফোনের অবস্থা আর সাথে কী কী আছে (charger, case, SIM) লিখে রাখে। তারপর inspect করে।
5. **Final quote:** technician আসল দাম পাঠায় — labour, parts, extra আলাদা করে। রহিম approve করে বাকি টাকা দেয় (quote থেকে আগের inspection fee বাদ দিয়ে), অথবা decline করে।
6. **Repair:** technician progress update দেয়, শেষে চাইলে ছবি দিয়ে প্রমাণ রাখে।
7. **ফেরত:** ফোন ফেরত পেলে রহিম confirm করে। ৩ দিনেও confirm না করলে নিজে থেকেই confirm হয়ে যায় (যদি না রহিম বলে "পাইনি")। তখন technician পায় ৯০%, Sarabo রাখে ১০%।
8. **Feedback ও warranty:** রহিম ১–৫ star review দেয়। ৩০ দিনের মধ্যে একই সমস্যা আবার হলে warranty claim করতে পারে।

```
Request → Offers → Choose + inspection fee → Pickup → Inspection → Quote → Approve + pay
→ Repair → Handover (90/10 split) → Review → (৩০ দিনের warranty)
```

যার কাছে tracking code (`SRB-…`) আছে, সে login ছাড়াই একটা public page-এ timeline দেখতে পারে — কিন্তু সেখানে কারো নাম, email বা payment-এর তথ্য দেখায় না।

৪. টাকার নিয়মগুলো (এগুলো পরিষ্কার জানতে হবে)

বিষয়	নিয়ম
Inspection fee	technician offer-এ নিজে ঠিক করে, তবে কমপক্ষে ১০০ টাকা; estimate-এর max ১,০০০ টাকার বেশি হলে কমপক্ষে ২০০। এই fee পরে final quote থেকে বাদ যায় — মানে customer-কে দুবার দিতে হয় না।
Commission	technician ৯০%, Sarabo ১০%, পুরো quote total-এর উপর, পূর্ণ টাকায় round করে।
Cancel	pickup-এর অন্তত ২ ঘণ্টা আগে cancel করলে inspection fee পুরো ফেরত; পরে করলে fee রাখা হয় (সেটাও ৯০/১০ ভাগ)।
Quote decline / না দেওয়া	approve করেও ৩ দিনে টাকা না দিলে request বন্ধ হয়ে যায়; inspection fee রাখা হয়, technician free হয়ে যায়।
Withdrawal	technician wallet থেকে bKash withdrawal চায়, admin bKash Send Money করে TrxID লিখে রাখে (এটা manual)।
Warranty refund	repair আবার ঠিক না হলে admin পুরো quote total ফেরত দেয়; technician-এর settlement উল্টে যায়। আগেই টাকা তুলে নিলে সেটা "owed" দেখায়, পরের আয় থেকে কাটা যায়।

Payment সম্পর্কে সৎভাবে বলবে: Stripe এখন **test mode**-এ চলছে, কারণ বাংলাদেশের business-কে Stripe local account দেয় না। আসল launch-এ SSLCOMMERZ বা bKash Payment Gateway লাগবে — সেটা একই payment service-এর পেছনে বসানো যাবে।

৫. Smarter matching — কোন offer "Best match"?

আগে offer গুলো শুধু যে আগে এসেছে সেই order-এ দেখাতো। এখন ৫টা জিনিস দেখে score দেয়, সবই Sarabo-র নিজের data থেকে (কোনো paid service বা map API লাগে না):

Factor	Weight	কীভাবে
Rating	৩০%	review-এর গড়, কিন্তু ৩★-এর দিকে "smooth" করা — যাতে একটা মাত্র ৫★ review চল্লিশটা ৪.৮★-কে হারাতে না পারে।
Reliability	২৫%	কয়টা repair শেষ করেছে বনাম কয়টা pickup miss করেছে।
Price	২০%	অন্য offer-গুলোর তুলনায় estimate কত।
District	১৫%	customer-এর একই district-এ কিনা।
Reply time	১০%	job post হওয়ার কতক্ষণ পরে সাধারণত offer দেয় (শেষ ২০টা offer-এর median)।

নতুন technician, যার কোনো history নেই, সে neutral score পায় — শাস্তিও না, বোনাসও না। আর শেষ সিদ্ধান্ত customer-এর; system নিজে কাউকে book করে না।

৬. AI কোথায় কোথায় আছে?

Google Gemini ব্যবহার করেছি, ৪ জায়গায়:

1. **Customer support chat** — site নিয়ে প্রশ্নের উত্তর দেয়।
2. **Request helper** — customer বাংলা, English বা Banglish-এ লিখলে বা ছবি দিলে AI form-এর draft বানিয়ে দেয় (কোন device, কোন service)।
3. **Offer note** — technician-কে offer-এর note লিখতে সাহায্য করে, সাথে একটা price guide দেখায়।
4. **Admin report summary** — report-এর সংখ্যা দেখে সহজ ভাষায় summary লেখে।

AI-কে বিশ্বাস না করে guard-rail রেখেছি (viva-তে এটা জোর দিয়ে বলবে):

- AI শুধু live catalogue-এ যে service id আছে সেখান থেকেই বাছতে পারে, নিজে কিছু বানাতে পারে না।
- AI কখনো দাম ঠিক করে না।
- AI-এর লেখা থেকে phone নম্বর/email-এর মতো contact details মুছে ফেলা হয়।
- Admin summary-তে এমন কোনো বাক্য থাকলে যেখানে আসল data-তে নেই এমন সংখ্যা আছে — সেই বাক্যটা বাদ দেওয়া হয়। মানে AI "বানিয়ে" সংখ্যা বলতে পারে না।
- প্রতি user আর পুরো site-এর জন্য দৈনিক limit, আর একটা model fail করলে fallback model।

৭. Technology — কী দিয়ে বানানো আর কেন

অংশ	কী ব্যবহার করেছি	কেন (সহজ কথায়)
Frontend	React 19, Vite, React Router, TanStack Query, Tailwind CSS, Radix UI	single-page app, দ্রুত; TanStack Query server-এর data cache করে আর update হলে নিজে refresh করে।
Backend	Node.js + Express 5 (REST API)	JavaScript দুই দিকেই, সহজ আর হালকা।

Database	MongoDB Atlas	flexible document; আর সবচেয়ে জরুরি — **transaction** support আছে, টাকার কাজে লাগে।
Login	Firebase Authentication (+ server-এ Firebase Admin)	email/password আর Google login ready; server প্রতিটা request-এ token verify করে।
Payment	Stripe Checkout + webhook	নিরাপদ checkout page, webhook দিয়ে নিশ্চিত হওয়া যায় টাকা সত্যিই এসেছে।
ছবি রাখা	Supabase Storage (private bucket)	ছবি public না; অল্প সময়ের signed URL দিয়েই শুধু দেখা যায়।
Email	Nodemailer + Gmail	free, সহজ।
Hosting	Vercel (client + API), Vercel Cron	push করলেই deploy; cron দিয়ে প্রতিদিন reminder আর auto-close।
Testing / CI	Server integration test, Playwright, GitHub Actions	প্রতিটা push-এ সব test নিজে থেকে চলে।

Architecture এক নজরে

```
Browser (React app, Vercel)
   │  Firebase ID token যায় Authorization header-এ
   ▼
Express API (Vercel, Mumbai region) ──► MongoDB Atlas (Mumbai)
   ├─► Firebase Admin  (token যাচাই)
   ├─► Stripe          (payment, webhook, refund)
   ├─► Supabase        (private ছবি)
   ├─► Google Gemini   (AI)
   └─► Gmail SMTP      (email)
Vercel Cron → প্রতিদিন pickup reminder, পুরনো কাজ auto-close
```

একটা request server-এ ঢুকলে ক্রম: **route → middleware (token, role যাচাই) → controller (এই অবস্থায় এই কাজ চলবে কিনা) → service (আসল নিয়ম, transaction) → database**।

৮. Security আর data ঠিক রাখা — আমার project-এর শক্ত দিক

- **Transaction:** booking, approve, complete, settlement, refund — যেখানেই টাকা বা assignment বদলায়, সব একটা MongoDB transaction-এ। মাঝপথে কিছু fail করলে সব আগের মতো থাকে, অর্ধেক কাজ হয়ে পড়ে থাকে না। একই technician-কে দুজন একসাথে book করতে পারে না।
- **Webhook-ই সত্য:** "টাকা দেওয়া হয়েছে" — এটা browser বললে বিশ্বাস করি না, Stripe-এর signature-যাচাই করা webhook বললে তবেই।
- **Idempotency key:** refund দুবার চলে গেলেও টাকা দুবার ফেরত যায় না। refund fail করলে "pending" থাকে, আবার চেষ্টা করা যায়।
- **Role ও state check server-এ:** UI-তে button লুকানো যথেষ্ট না; server প্রতিটা কাজে দেখে তুমি কে আর request-টা এখন কোন অবস্থায়। ১৪টা status, ভুল ধাপে যাওয়া যায় না।
- **অন্যান্য:** rate limiting (MongoDB-তে রাখা, তাই serverless-এ সব instance মিলে কাজ করে), CORS allow-list, ১২৮-bit random tracking code (অনুমান করা যায় না), private ছবি, log-এ request body রাখা হয় না।

৯. কোনো request যেন আটকে না থাকে

পুরো workflow ঘুরে দেখেছি কোথায় একটা request চিরকাল অপেক্ষায় পড়ে থাকতে পারে, আর প্রতিটার একটা রাস্তা বানিয়েছি:

- quote approve করে টাকা দেয় না → ৩ দিনে auto close।
- customer receipt confirm করে না → ৩ দিনে auto confirm (আপত্তি থাকলে admin-এর কাছে যায়)।
- customer pickup-এর সময় বাড়িতে নেই → technician জানায়, customer নতুন সময় দেয়; এটা technician-এর দোষ ধরা হয় না।
- paid repair শেষ করা যাচ্ছে না → admin cancel করে পুরো টাকা refund করে।
- কাজ নড়ছে না → ৩ দিন (চলমান repair-এ ১৪ দিন) পর admin-এর overview-তে alert।

১০. Testing আর deploy

- **২,৭৬১টা server test** — আসল HTTP request আর আসল MongoDB replica set দিয়ে (Stripe/AI/storage fake)।
- **৫৮টা Playwright browser test** — public page গুলোর জন্য।
- GitHub Actions-এ প্রতিটা push-এ lint, build আর test চলে, একটা আলাদা throwaway database-এ।
- Client আর API দুটোই Vercel-এ live: `sarabo-client-six.vercel.app`।

১১. কাজ করতে গিয়ে যে সমস্যায় পড়েছিলাম (গল্প হিসেবে বলা যায়)

সমস্যা	কারণ	সমাধান
Site খুব slow, প্রথম request-এ ১০ সেকেন্ডেরও বেশি	Database Mumbai-তে, কিন্তু API চলছিল আমেরিকায় (Washington)। প্রতিটা DB call পৃথিবী ঘুরে আসছিল।	`vercel.json`-এ এক লাইন — API-কে Mumbai region-এ নিলাম। ১০.৭ সেকেন্ড → ২.৯, সাধারণ request ০.৭৫ → ~০.২ সেকেন্ড।
Robi mobile data-তে site খোলে না	খুঁজে দেখি সমস্যা DNS-এ — `.app` / `.dev` address resolve হচ্ছিল না, code-এ না।	Phone-এর DNS `dns.google` দিলে চলে। স্থায়ী সমাধান নিজের `.com` domain — future work-এ রেখেছি।
AI support মাঝে মাঝে fail করতো	পুরনো Gemini model free tier-এ ৬–১৬ সেকেন্ড নিতো, server-এর ১২ সেকেন্ডের limit পার হয়ে যেত; মাঝে মাঝে 503/429 দিতো।	দ্রুত model (`gemini-3.5-flash-lite`, ১–৩ সেকেন্ড) default করেছি; প্রথমটা fail করলে দ্বিতীয় চেষ্টা যায় fallback model-এ।
কিছু request আটকে যেতে পারতো	কিছু ধাপে "এরপর কী" ঠিক করা ছিল না।	উপরের ৯ নম্বর অংশের সব নিয়ম।

আরেকটা বড় কাজ ছিল **migration**: project শুরু হয়েছিল একটা parcel-delivery codebase ("zap_shift") থেকে। ধাপে ধাপে database, collection, API route আর UI-এর ভাষা repair-এর দিকে বদলেছি — প্রতিটা script আগে dry-run, তারপর আসল run।

১২. দুর্বলতা আর ভবিষ্যতে কী করা যায় (সৎভাবে বলবে)

এখনকার limitation	ভবিষ্যতে

Stripe test mode, technician payout manual (bKash Send Money)	SSLCOMMERZ / bKash gateway, auto payout
একই slot-এর শেষ জায়গা দুজন একই সেকেন্ডে নিলে দুজনই পেতে পারে	atomic slot counter
একজন technician একসাথে একটাই active job নিতে পারে	technician scheduling, দৈনিক capacity
দামগুলো demo-র জন্য, আসল দোকানের সাথে ঠিক করা না	আসল business-এর সাথে দাম ঠিক করা
Browser test signed-in page cover করে না (server test করে)	test account দিয়ে signed-in browser test
এখনো আসল customer নেই, load test হয়নি	pilot launch, load test, security audit
UI শুধু English (AI বাংলা input নেয়)	পুরো বাংলা interface, SMS notification, mobile app/PWA, live technician location, cash payment

১৩. মুখস্থ রাখার সংখ্যা

কী	মান	কী	মান
Service	৪৫টা, ৮ ধরনের device	Request status	১৪টা
Commission	৯০ / ১০	Min inspection fee	১০০ (বা ২০০) টাকা
Cancel-এ পুরো refund	pickup-এর ≥২ ঘণ্টা আগে	Warranty	৩০ দিন, max ২ claim
Pickup slot	দিনে ৪টা, ২ ঘণ্টার; শুক্রবার বন্ধ	Damage photo	max ৩টা
Matching weight	৩০/২৫/২০/১৫/১০	API	১২৫টা route
Server test	২,৭৬১	Browser test	৫৮
Code size	client ~২৮.৭k লাইন, server ~৪৫.১k লাইন	Report	১১১ পৃষ্ঠা

১৪. ২ মিনিটে project বোঝানোর script

> "আমার project-এর নাম **Sarabo** — একটা electronics repair service management platform। আমাদের দেশে phone বা laptop নষ্ট হলে আমরা পরিচিত দোকানে দিই, আগে দাম জানি না, device কোথায় আছে দেখতে পাই না, আর কোনো record থাকে না।
>
> Sarabo-তে customer অনলাইনে request দেয়, ছবি আর pickup-এর সময় দেয়। ওই এলাকার verified technician-রা offer পাঠায়, আর system rating, reliability, দাম, এলাকা আর reply time দেখে best match দেখায়। customer একজনকে বেছে inspection fee দেয়, technician device নিয়ে inspect করে final quote দেয়, customer approve করে টাকা দেয়, repair হয়, device ফেরত আসে। পুরো সময় customer প্রতিটা ধাপ দেখতে পায়, আর ৩০ দিনের warranty থাকে। technician পায় ৯০%, platform রাখে ১০%।
>
> এটা React আর Express দিয়ে বানানো, database MongoDB, login Firebase, payment Stripe, আর ৪ জায়গায় Google Gemini AI আছে — কিন্তু guard-rail সহ, AI কখনো দাম ঠিক করে না। টাকার সব কাজ database transaction-এ হয়, তাই মাঝপথে fail করলেও data ভুল হয় না। ২,৭৬১টা server test আর ৫৮টা browser test আছে, প্রতিটা push-এ চলে, আর পুরো system Vercel-এ live।"

মনে রাখার কৌশল: প্রশ্ন যাই আসুক, আগে এক লাইনে উত্তর, তারপর একটা উদাহরণ (রহিমের ফোন), তারপর পারলে একটা edge case। না জানলে বানিয়ে বলবে না — "এটা এখন নেই, limitation-এ রেখেছি।" বিস্তারিত code প্রশ্নের জন্য `Sarabo_Viva_QnA.pdf` দেখো।